

A Lambda Architecture for Real-Time Ride-Hailing Fleet Operations

EC8203 Applied Big Data Engineering — Mini Project (25%)

Use Case 1: Ride-Hailing Fleet Operations

September 30, 2026

Summary. This report documents a complete, running data platform that ingests continuous vehicle telemetry from Kafka and a once-per-day vehicle expense file, processes both, and answers a two-part business question: *what is fleet utilisation and earnings by zone right now*, and *which vehicles became unprofitable once yesterday's fuel and maintenance costs are applied*. The two halves have different latency and correctness contracts, which is the reason a **Lambda** architecture was selected over Kappa. The system runs as fifteen Compose services, including one-shot initialization jobs; simulated time is compressed so that one simulated day elapses every five real minutes.

Contents

1	Use Case and Business Requirements	2
1.1	The scenario	2
1.2	Requirements interpreted from the brief	2
1.3	Deliverables of the running system	3
2	Architecture Decision: Lambda versus Kappa	3
2.1	The two candidates	3
2.2	Decision	3
2.3	Justification against the four decision axes	3
2.4	Rejected alternatives	4
2.5	Trade-offs accepted, honestly	4
3	System Architecture	5
3.1	The master dataset is the pivot	5
3.2	Simulated clock	5
4	Technology Stack and Justification	6
5	Data Ingestion	7
5.1	Streaming source	7
5.2	Daily-batch source	8
6	Processing	8
6.1	Speed layer	8
6.2	Batch layer	9
6.3	Orchestration	9
7	Storage and Serving	10
7.1	Schema	10
7.2	Serving layer and the merge	10

8	Observability Design	10
8.1	Structured logging — across all stages	11
8.2	Tracing — correlation across stages	11
8.3	Metrics	11
8.4	Alerting	11
9	Results	12
9.1	Verification	12
9.2	Observed speed/batch divergence	12
9.3	Testing	13
10	Limitations and Production Scale	13
10.1	Limitations of what was built	13
10.2	What would change at production scale	13
11	Conclusion	14

1 Use Case and Business Requirements

1.1 The scenario

A ride-hailing operator runs a fleet of vehicles across a city. Two groups inside the company need answers from the same underlying activity, but they are not the same question:

- **Dispatch** needs to know, *now*, how many vehicles are working, how many are idle, where earnings are being generated, and which vehicles have been sitting still too long. A number that is ten seconds old and approximately right is useful. A number that is perfectly right and twenty minutes old is not.
- **Fleet finance** needs to know which vehicles stopped paying for themselves. That calculation cannot be made until the fuel partner and the garage submit the day's costs, which happens once per day, after the fact. This number decides whether a vehicle is withdrawn from service or a driver's contract is reviewed, so "approximately right" is not acceptable.

1.2 Requirements interpreted from the brief

Table 1 restates the use case as engineering requirements. The split down the middle is the finding that drives every subsequent decision.

Table 1: The business question decomposed. The two columns have incompatible contracts.

	Live operational view	Daily reconciliation
Question	Utilisation and earnings by zone, now	Which vehicles are unprofitable
Latency budget	Seconds	Once per day, minutes to produce
Accuracy needed	Directional; approximation acceptable	Exact; it is money
Data completeness	Partial window is fine	Whole day, including late events
Sources required	Telemetry stream only	Telemetry <i>and</i> the expense file
Correction path	Superseded by the next window	Must be restatable on dispute
Consistency model	Eventual	Strong, per day
Cost of a wrong answer	Dispatcher re-checks	A vehicle wrongly withdrawn

1.3 Deliverables of the running system

Following the use case’s suggested outputs, the platform produces: (i) an API returning real-time fleet utilisation metrics — active vehicles, idle ratio, trips, earnings by zone; (ii) threshold-based alerts when a vehicle has been idle beyond a configurable limit; and (iii) a daily per-vehicle profitability reconciliation report, rendered as a standalone HTML page and a CSV, and also queryable through the API. A live dashboard presents both views together.

2 Architecture Decision: Lambda versus Kappa

2.1 The two candidates

Kappa treats the log as the system of record and runs a single stream-processing path. History is not a separate store; it is the log replayed from an earlier offset, usually by a second instance of the job writing to a new output table. There is one codebase, one execution model, one set of semantics to reason about.

Lambda keeps an append-only *master dataset* and derives two views from it: a *speed layer* that computes fast, approximate answers from the live stream, and a *batch layer* that periodically recomputes exact answers from the complete data. A serving layer presents both.

2.2 Decision

This project implements a Lambda architecture. The determining factor is not latency — both architectures meet the latency budget comfortably — but that the authoritative answer requires a *whole day* of data joined to an *externally supplied, revisable* file, while the operational answer requires an incomplete window *now*. These are two different correctness contracts over the same events, and Lambda is the architecture that makes that difference explicit instead of hiding it.

2.3 Justification against the four decision axes

Latency. Dispatch needs seconds; the reconciliation needs a daily cadence. The speed layer triggers every 10–15 seconds, comfortably inside the operational budget, and Kappa would meet it equally well. *Latency does not discriminate between the two architectures here*, and any argument that claims it does is reaching for the wrong reason.

Replay and recomputation. This axis does discriminate. Kappa’s recomputation story is “replay the log”, which binds the recomputation horizon to Kafka’s retention. Two properties of this use case make that uncomfortable. First, a billing dispute over fuel costs can be raised long after the fact, so the horizon over which the operator must be able to recompute is months, not days — retaining months of raw high-frequency GPS telemetry in Kafka is possible but is the most expensive way to store cold data. Second, recomputation here is not only about fixing a bug in the job; it is about *restating a day* because the partner reissued a corrected invoice. Lambda’s answer is direct: the day’s Parquet partition is still there, the corrected file replaces the old one, and the DAG re-runs that date. In this implementation every write is an upsert keyed on (`sim_date`, `vehicle_id`), so a restatement replaces the day rather than duplicating it.

Cost. At the scale of this demonstration, cost is not a differentiator. At realistic scale it is, and it favours Lambda for this workload: telemetry is high-volume and mostly cold after the day closes, so it belongs in partitioned columnar files on object storage — where a daily job reads one date’s partition with predicate pushdown — rather than in broker storage sized for replay. The countervailing cost of Lambda is engineering time for two code paths, addressed below.

Consistency. The decisive axis, together with replay. The daily file is not a stream and does not behave like one: it arrives once, it is complete only when the file is closed, rows may be missing, and it may be *corrected and reissued*. That is an update-in-place artifact, whereas a log is append-only. To handle it in Kappa one either shreds the file into synthetic events — and then must invent semantics for a correction that supersedes earlier events — or performs a stream–static join against a table that is itself mutable, which quietly reintroduces batch semantics without admitting it. Lambda has a natural place for this: the batch layer joins two complete datasets at rest and produces a result that is correct for that day and re-derivable.

2.4 Rejected alternatives

Table 2: Alternatives considered and why each was rejected.

Alternative	What it would look like	Why rejected
Kappa	One Structured Streaming job; the daily file shredded into events onto a second topic; reconciliation as a stream–static join; history as log replay.	Requires inventing supersede semantics for corrected invoices, binds the recomputation horizon to broker retention, and makes the money figure a product of watermarked, approximate aggregation. The machinery exists only to make a batch source look like a stream.
Pure batch (no speed layer)	Land telemetry in the lake; run everything on a schedule.	Answers only half the business question. Dispatch cannot act on figures computed after the fact; the idle-vehicle alert is worthless if it fires the next morning.
Pure streaming, no batch layer	Speed layer alone; stream–static join against the expense table; windowed aggregates serve as the daily report.	Events later than the watermark are dropped permanently, and <code>approx_count_distinct</code> becomes the trip count a driver is paid on. Neither is defensible for a financial figure, and neither can be restated.
Managed cloud services	Kinesis/Pub-Sub, Dataflow or Databricks, BigQuery/Redshift.	Sound production choice and the natural migration path, but it cannot be submitted as a self-contained, reproducible artefact that a marker runs locally with one command. Noted in §10 as the production direction.

2.5 Trade-offs accepted, honestly

Lambda’s standard criticism — famously Kreps’ original argument for Kappa — is that the same business logic is implemented twice, in two engines, and the two implementations drift. That criticism is legitimate and is only *partly* mitigated here:

- **Mitigated:** validation rules (`common/validation.py`), zone assignment (`common/geo.py`), the vehicle roster, and the economic thresholds (`batch_layer/profitability.py`) are single modules imported by both layers. A rule change happens once. Both layers also run on the same engine, Spark, so there is no second dialect to maintain.
- **Not mitigated:** the aggregation logic itself genuinely differs — the speed layer windows over event time with a watermark and HyperLogLog distinct counts, while the batch layer

groups an entire day exactly. That is not accidental duplication that better factoring would remove; it is the intended difference between the two answers.

- **Residual risk:** the dashboard can show a live figure that a later report contradicts. Rather than hide this, every API response is labelled ("view": "speed", "accuracy": "approximate" versus "view": "batch", "accuracy": "authoritative"), and `/api/lambda/compare/<date>` reports the difference between the two as a number. Section 9 gives the measured gap.
- **Storage duplication:** the same events exist in Kafka (briefly), in Parquet (durably) and in aggregated form in Postgres. This is inherent to Lambda and is accepted.

3 System Architecture

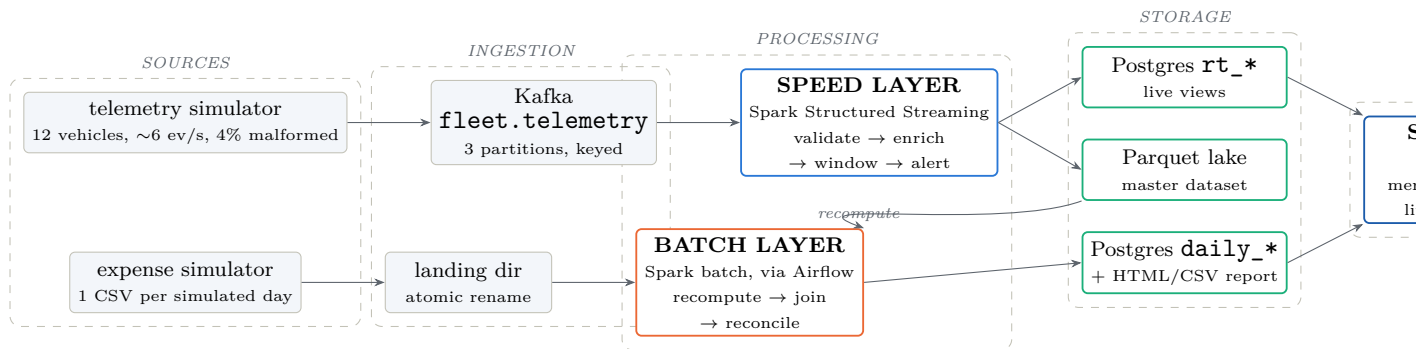


Figure 1: End-to-end architecture. Blue is the speed path, orange the batch path, green storage. The master dataset is written by the speed layer and *read back* by the batch layer — the defining loop of a Lambda architecture.

3.1 The master dataset is the pivot

The single most important structural decision is that the speed layer writes *every parsed event to Parquet, including the ones it rejected*, and the batch layer reads that back rather than consuming Kafka itself. Two consequences follow:

1. The batch layer can *revise a decision the stream made*. If a validation rule is later found to be too strict, the rejected rows are still in the master dataset with their violation reasons attached, and yesterday can be recomputed under the corrected rule. Had the speed layer filtered before writing, that data would be gone.
2. There is exactly one durable copy of raw history, and its retention is a storage decision rather than a broker configuration.

3.2 Simulated clock

As permitted by the brief, simulated time is compressed. **One simulated day elapses every 300 real seconds**, a factor of $86400/300 = 288$:

Real time	Simulated time	What happens
1 second	4.8 minutes	~6 telemetry events
12.5 seconds	1 hour	one windowed aggregation closes
5 minutes	1 day	one expense file; one Airflow reconciliation

Every downstream quantity is expressed in *simulated* time — the 60-minute window, the 30-minute watermark, the 45-minute idle-alert threshold, the report date — so each keeps its real-world meaning; only the wall-clock rate changes. All components must agree on where the calen-

dar starts, so the first process to boot writes `/data/_sim_anchor.json` atomically (`O_CREAT|O_EXCL`) and every other process reads it. The compression factor is configurable via `SIM_DAY_SECONDS`.

4 Technology Stack and Justification

Each choice below is argued from a constraint of *this* use case rather than general popularity, and each names what was rejected.

Table 3: Technology selection.

Layer	Choice	Justification and rejected alternative
Ingestion	Apache Kafka 7.6.1, topic <code>fleet.telemetry</code> , 3 partitions, keyed by <code>vehicle_id</code>	Three properties are needed and Kafka has all three: <i>partition-keyed ordering</i> , so one vehicle’s events stay ordered while three consumers work in parallel — the “latest state per vehicle” logic depends on it; <i>retention</i> , so the speed layer can be restarted and resume without data loss; and <i>fan-out</i> , so the quarantine stream is a first-class topic other consumers can subscribe to. <i>Rejected:</i> Redis Streams and RabbitMQ — adequate as transport, but neither offers durable partitioned replay, which is what makes restart-safety cheap here.
Stream processing	pro- Spark Structured Streaming 3.5.1	Chosen over Storm principally because <i>the batch layer needs an engine too</i> , and using Spark for both means the two layers share a language, a DataFrame API and the modules in <code>common/</code> — directly reducing Lambda’s duplication penalty. Structured Streaming also provides event-time windowing and watermarking natively, which this use case needs because event time is simulated and diverges from arrival time. <i>Rejected:</i> Apache Storm — genuinely lower per-event latency, but our budget is seconds, not milliseconds, so that advantage buys nothing while costing a second engine for the batch layer and a heavier local deployment.
Orchestration	Apache Airflow 2.8.4, LocalExecutor	The daily job is not a single command on a timer; it is a dependency graph — <i>wait</i> for the file, <i>run</i> the job, <i>validate</i> what it wrote, <i>mark</i> the day complete — with retries, visible run history and the ability to re-run one date. Cron expresses none of that. Airflow’s sensors and per-task retries map exactly onto this shape.

Layer	Choice	Justification and rejected alternative
Master dataset	Parquet on the shared volume, partitioned by <code>sim_date</code>	The batch layer’s access pattern is “read exactly one date, project a dozen columns”, which is the pattern columnar storage with partition pruning is built for. Immutable append also matches Lambda’s requirement that the master dataset be the durable record. <i>Rejected</i> : keeping raw history in Kafka (expensive for cold data) or in Postgres rows (no columnar scan; the table grows without bound).
Serving store	PostgreSQL 15	Serving queries are small — tens of vehicles, a few hundred window rows — but their <i>shape</i> is unpredictable: filters, joins between the live and daily tables, ordering by profit, <code>COUNT(*) FILTER</code> . That is SQL’s strength. Equally important, <code>INSERT ... ON CONFLICT DO UPDATE</code> makes every write from both layers idempotent, which is what makes at-least-once delivery safe. <i>Rejected</i> : Cassandra — built for very high write throughput against known partition keys with no joins; we have neither the volume to need it nor the fixed query shapes it demands, and it would make idempotent upserts and ad-hoc reporting harder, not easier.
Serving API	FastAPI + Uvicorn	Generates the OpenAPI documentation the demo uses, integrates with <code>prometheus_client</code> in a few lines, and its middleware hook is where per-request metrics and structured logs are attached.
Monitoring	Prometheus, Pushgateway, Grafana	Pull-based scraping suits containers that come and go. The Pushgateway exists specifically because the batch job lives ~30 seconds and can never be scraped — it pushes, grouped by <code>sim_date</code> , so re-running a day replaces that day’s sample. Alert rules live in version control as code.
Packaging	Docker Compose, 15 services	Reproducibility is explicitly assessed. Compose builds and starts the platform; named volumes preserve simulation data across ordinary stops. <code>docker compose down -v</code> deletes those volumes and is only for an intentional full reset.

5 Data Ingestion

5.1 Streaming source

`simulators/telemetry_producer.py` simulates twelve vehicles, each running an `idle` → `enroute` → `on_trip` → `idle` state machine and emitting position, speed, status and accrued fare every two real seconds. Design points that matter:

- **Events are keyed by `vehicle_id`**, so all of a vehicle’s events hash to one partition and stay mutually ordered. Without this the “latest state” upsert would be a race.

- **Two timestamps per event.** `event_time` is on the simulated clock and is the event-time axis for windowing; `ingest_time` is the real wall clock and exists to measure pipeline lag. Conflating them would make lag unmeasurable.
- **Fare is carried two ways.** `fare` is the running cumulative total of the current trip; `fare_increment` is what accrued since the vehicle's previous event. The speed layer sums increments so a live earnings figure exists mid-trip; the batch layer takes the final cumulative fare of each completed trip as authoritative revenue. *The two deliberately disagree slightly*, and that disagreement is the Lambda trade-off made visible.
- **About 4% of events are deliberately malformed** — null position, negative speed, unknown status, a fare accruing while parked. Telemetry from cheap in-vehicle units is not clean, and the validation and quarantine path needs real work to do. A unit test asserts that *every* injected anomaly is actually rejected, so the quarantine path cannot silently pass dirty data.
- **An event_id is minted at the source** and never rewritten. It is the correlation id used for tracing (§8).

5.2 Daily-batch source

`simulators/expense_batch_source.py` writes one CSV per simulated day (`vehicle_id`, `fuel_cost`, `maintenance_cost`, `distance_covered`, `service_flag`). Two properties were built in deliberately:

- **Atomic publication.** The file is written to a `.tmp` name and then `os.replaced` into position. Rename within a filesystem is atomic; a streaming write is not. Without this, Airflow's sensor could observe — and the batch layer could parse — a half-written file. A test asserts no `.tmp` file survives publication.
- **Realistic imperfection.** Roughly 5% of vehicles are missing from the file (the partner did not report them) and about 10% report a distance that disagrees sharply with the GPS track. Both are situations the batch layer must survive: the join is a left join so a missing row does not delete a vehicle from the report, and the distance disagreement is precisely what the reconciliation exists to surface.

Both simulators build their vehicle list from the same deterministic roster in `common/fleet.py`, seeded with a fixed value. This is what makes the two feeds describe the *same* fleet rather than two unrelated sets of random numbers, and it makes a demo reproducible.

6 Processing

6.1 Speed layer

Two Structured Streaming queries run in one application, so that a slow sink in one cannot stall the other's watermark.

Query ingest-and-state (10 s trigger). Parses the JSON payload against an explicit schema, applies the validation rules as a UDF wrapping `common/validation.py`, enriches valid rows with an operating zone (a 3×2 grid over the city bounding box), then performs four side effects inside one `foreachBatch`: appends every row to the Parquet master dataset; republishes rejected rows to `fleet.telemetry.quarantine` with their violation reasons; upserts each vehicle's newest state into Postgres; and raises idle alerts.

Query zone-windows (15 s trigger). Tumbling 60-simulated-minute windows per zone with a 30-simulated-minute watermark, producing events, active vehicles, idle ratio, trips, earnings and mean speed, upserted on (`window_start`, `zone`).

Three implementation decisions worth defending.

1. **Approximation is deliberate, not a shortcut.** `approx_count_distinct` (HyperLogLog) is used for vehicle and trip counts. Exact `countDistinct` is not supported in a streaming aggregation, and a fast approximate answer is exactly the speed layer's contract. The batch layer counts the same quantities exactly.
2. **Idle state is maintained in SQL, not in Spark state.** The `idle_since` column is set by a `CASE` inside the upsert: it is preserved while the vehicle remains idle, cleared the moment it moves. An idle *spell* therefore keeps its start time across micro-batches, and “idle for 45 minutes” measures the spell rather than the batch. Alerts become a query, and a unique constraint on (`alert_type`, `vehicle_id`, `idle_since`) yields one alert per spell instead of one per trigger. This was chosen over `flatMapGroupsWithState` because the alerting rule needs to be queryable by the serving layer anyway.
3. **The upsert refuses to move backwards.** The statement carries `WHERE EXCLUDED.last_event_time >= rt_vehicle_state.last_event_time`, so a replayed or out-of-order micro-batch cannot rewind the live view — necessary because delivery is at-least-once.

6.2 Batch layer

`batch_layer/daily_reconciliation.py` recomputes a simulated day from scratch:

- **Revenue** = the final cumulative fare of every completed trip, computed as `max(fare)` grouped by (`vehicle_id`, `trip_id`). `max` rather than “last event” is deliberate: the master dataset is a union of micro-batches with no global ordering, so an order-dependent definition would not be reproducible.
- **Distance** = haversine over consecutive GPS fixes, using a window function partitioned by vehicle and ordered by event time. Segments longer than 25 km between consecutive fixes are discarded as bad fixes rather than allowed to dominate the day.
- **Utilisation** = the share of a vehicle's events not spent idle.
- **Join** — left joins throughout, on `vehicle_id`. A vehicle that drove but was never invoiced, and one invoiced but never seen, are both real situations that must appear in the report rather than vanish from it.
- **Reconciliation** — profit, margin, revenue per km, and the percentage variance between GPS distance and billed distance. Vehicles are flagged *unprofitable* (margin below 10%) or *distance disputed* (variance above 20%).

The job reads its expense CSV with an *explicit* schema rather than `inferSchema`: a day in which every maintenance cost happened to be a whole number would otherwise arrive as integers and silently change the arithmetic. Every write is an upsert keyed on (`sim_date`, `vehicle_id`), so re-running a day — the correction path argued for in §2 — replaces it rather than doubling it.

6.3 Orchestration

The DAG `fleet_daily_batch` runs every five real minutes (one simulated day):

```
find_pending_day → wait_for_master_dataset → run_batch_layer → validate_output →
mark_processed → announce
```

It deliberately does *not* key off Airflow's logical date. It discovers work by finding an expense file with no completion marker, which means a failed run retries the same day, a late file is still processed, and re-running a day is a deliberate, safe act (delete the marker). Ticks with nothing to do end as *skipped*, which is the intended outcome on most five-minute boundaries.

`validate_output` is a genuine quality gate, not decoration: it checks the audit row reports success, that a fleet summary exists, that revenue and cost are non-negative, that no vehicle row

has a null profit, and that the HTML report was actually rendered. If any check fails the day is left unmarked and the run fails visibly, rather than a bad report reaching the dashboard.

7 Storage and Serving

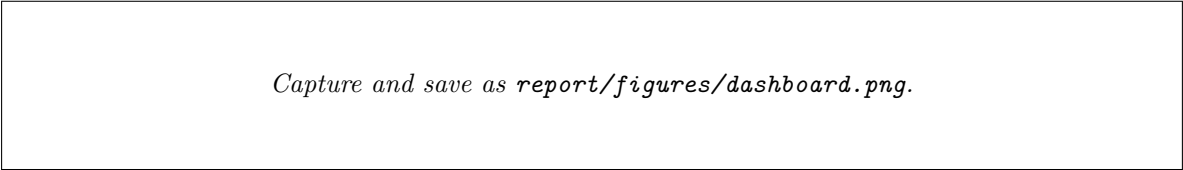
7.1 Schema

The serving store separates the two views by naming convention, because a reader must be able to tell them apart: `rt_*` tables are speed-layer output (approximate, seconds old, continuously overwritten); `daily_*` tables are batch-layer output (authoritative, one simulated day behind); `batch_run_audit` records provenance — which run produced which numbers, how long it took, and how many rows it read.

Every table has a natural primary key — `rt_vehicle_state(vehicle_id)`, `rt_zone_metrics(window_start, zone)`, `daily_vehicle_profitability(sim_date, vehicle_id)` — which is what allows both layers to write idempotently and makes at-least-once delivery safe in the serving store.

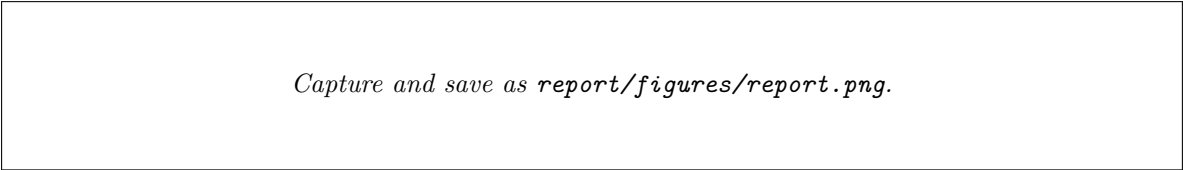
7.2 Serving layer and the merge

The serving layer's defining job in Lambda is not to expose the database; it is to present both views while keeping them distinguishable. Every response carries `view` and `accuracy` fields. `/api/fleet/vehicles/{id}` performs the merge explicitly, returning live state alongside the last authoritative day and a seven-day history. `/api/lambda/compare/{sim_date}` exists purely to quantify the residual gap between the two views — it is the endpoint that turns the architectural argument of §2 into a measured number.



Capture and save as `report/figures/dashboard.png`.

Figure 2: The live dashboard at `http://localhost:18000`: speed-layer tiles and per-window earnings above, the authoritative daily reconciliation below, and the measured speed/batch difference at bottom right.



Capture and save as `report/figures/report.png`.

Figure 3: The daily per-vehicle profitability reconciliation report, sorted worst-first, with unprofitable, disputed-distance and missing-expense-data vehicles flagged.

8 Observability Design

The guiding principle is that **data pipelines fail silently**. A crashed producer, a stalled stream and a batch job that never ran all look identical from the outside: no errors, no exceptions, just numbers that quietly stop moving. Consequently most alerting here is on *the age of the output*, not on error counts.

8.1 Structured logging — across all stages

Every component — both simulators, both Spark layers, the API, and the Airflow tasks — emits one JSON object per line through `common/logging_setup.py`, with a uniform shape: `ts`, `level`, `component`, `stage`, `event`, plus call-site context. `stage` is one of `ingest`, `process`, `store`, `serve`, `orchestrate`, so logs can be sliced by pipeline layer regardless of which process produced them. Key events include `micro_batch_complete` (with `events`, `clean`, `quarantined`, `vehicles_updated`, `ingest_lag_seconds`), `alert_raised`, `expense_file_published`, `batch_run_succeeded` and `batch_run_failed`.

8.2 Tracing — correlation across stages

Full distributed tracing (OpenTelemetry spans) was judged disproportionate for a pipeline whose hops are a Kafka topic and a Parquet write. Instead, three correlation identifiers make an end-to-end path reconstructable:

- **event_id** — minted per telemetry event at the source, carried into the Parquet master dataset and into any quarantine record. One event can therefore be followed from the producer log, through the quarantine topic, to its row in the lake.
- **Kafka coordinates** — `kafka_partition` and `kafka_offset` are stored with every row in the master dataset, so any row can be traced back to its exact position in the log.
- **run_id** — generated per batch run, written onto every output row, into `batch_run_audit`, and derived from the Airflow run identifier, so a number in the report traces back to the DAG run and its task logs.

8.3 Metrics

Every long-running component exposes `/metrics`; the batch job pushes to a Pushgateway because a 30-second process can never be scraped. What is measured, and why:

Stage	Metric	Why it exists
Ingest	<code>fleet_telemetry_events_total</code>	throughput, by vehicle status
Ingest	<code>fleet_telemetry_last_event_unixtime</code>	the freshness signal a silent stall shows up in
Ingest	<code>fleet_expense_last_file_unixtime</code>	detects a daily feed that stopped arriving
Process	<code>speed_events_total{outcome}</code>	clean vs quarantined, the data-quality ratio
Process	<code>speed_violations_total{rule}</code>	<i>which</i> rule is firing, for diagnosis
Process	<code>speed_batch_duration_seconds</code>	rising duration at a fixed trigger predicts falling behind
Process	<code>speed_ingest_lag_seconds</code>	real time between ingestion and processing
Process	<code>speed_batch_failures_total</code>	micro-batches that raised
Store	<code>speed_rows_written_total{sink}</code>	per-sink write volume
Batch	<code>fleet_batch_*</code> (pushed)	rows in/out, duration, revenue, profit, flags
Serve	<code>api_requests_total</code> , <code>api_request_duration_seconds</code>	rate, errors, duration
Serve	<code>api_speed_view_age_seconds</code> , <code>api_batch_view_age_seconds</code>	how stale each Lambda view is, as served

8.4 Alerting

Fifteen rules in `observability/alert_rules.yml`, grouped by layer (ingestion, speed layer, batch layer, serving). Representative examples:

- `NoTelemetryIngested` — no event published for 60s. The single most important rule: it is the one that distinguishes “healthy” from “frozen”.
- `SpeedLayerStalled` — no micro-batch completed for 90s, per query.

- **HighQuarantineRate** — more than 25% of the stream failing validation for 3 minutes. Some malformed telemetry is normal; a sustained quarter of it is an upstream device or schema problem.
- **BatchOutputStale** — no successful reconciliation for 15 minutes, i.e. three consecutive simulated days unreconciled.
- **BatchDroppingData** — fewer than half a day’s rows passed validation, meaning the report is built on a thin sample even though the job “succeeded”.
- **FleetUnprofitable** — a *business* alert on an ops dashboard by design: the fastest way to notice a pipeline that is healthy but producing nonsense.

Health checks complement the alerts: `/health` verifies database reachability rather than merely returning 200, and container health checks gate startup ordering in Compose.

Capture and save as `report/figures/grafana.png`.

Figure 4: Grafana “Fleet pipeline health”: ingestion rate and freshness, quarantine ratio, micro-batch duration and ingest lag, batch rows and reconciled profit, API rate and p95 latency, and the age of each Lambda view.

9 Results

9.1 Verification

The platform was run end to end on 2026-09-30. The current smoke check reported all seven checks as **PASS**: API/database health, live speed-state freshness, window aggregation, threshold alerts, batch reconciliation, speed/batch comparison, and monitoring (6/6 targets and 15 rules). The reconciled day at verification time was 2026-08-17. `scripts/smoke_check.py` also reports **PENDING** for checks that legitimately need more simulated time rather than treating them as failures.

9.2 Observed speed/batch divergence

For simulated date 2026-08-17, the compare endpoint returned speed-layer earnings of **LKR 54,177.15** over 24 windows and batch-layer revenue of **LKR 89,342.10** from 283 completed trips. The speed view was lower by **LKR 35,164.95**, or **39.36%** of batch revenue. The batch result was computed at 2026-09-30 16:45:18 Asia/Colombo. These are captured run values; a new run will produce different results.

The largest identified cause is a difference in fare definitions. The producer sets a LKR 120 flagfall directly on the cumulative fare when a trip starts, but the speed layer sums only `fare_increment`. That flagfall is therefore present in batch’s final fare per completed trip but absent from the stream’s increment sum. For 283 batch trips, the missing flagfall totals **LKR 33,960.00**. Subtracting it leaves a residual of **LKR 1,204.95**, about **1.35%** of batch revenue. This residual is measured but is not attributed solely to watermarking or HyperLogLog; event/window boundaries and the layers’ differing aggregation rules can also contribute. The speed trip count (492) is also not directly comparable to 283: the speed view adds approximate distinct counts across separate window/zone groups, so a trip crossing a group boundary can be counted more than once. The batch count is the authoritative completed-trip count for this daily report.

9.3 Testing

The repository contains **76 automated tests**, all of which passed in 3.23 seconds in the built application container. They require no broker, no Spark and no database, covering: the simulated clock's compression, day boundaries and shared-anchor race; zone assignment swept across the whole operating area; haversine against known distances; every validation rule individually and in combination; the profitability arithmetic and its threshold behaviour at the boundaries; the report renderer, including that it is self-contained and that missing values render as a dash rather than `None`; and that every deliberately corrupted event is actually rejected.

The suite found three real defects during development, the most instructive being a producer that computed a fare increment before applying the state transition, so a vehicle ending a trip emitted an `idle` event still carrying a fare — the producer was manufacturing its own quarantine traffic and inflating the data-quality metric. That is exactly the class of bug that is invisible in a running pipeline and obvious in a unit test.

10 Limitations and Production Scale

10.1 Limitations of what was built

- **Spark runs in local mode**, inside the speed-layer container and inside the Airflow worker. The code is unchanged on a cluster — the same `spark-submit` takes a different `-master` — but no distributed behaviour (shuffle across executors, node loss, speculative execution) is exercised.
- **Single broker, replication factor 1.** `acks=all` therefore means “the one replica has it”. Real durability needs three brokers and `min.insync.replicas=2`.
- **At-least-once, not exactly-once.** A crash between a sink write and the offset commit replays that micro-batch. Idempotent upserts make the *effect* at-most-once in the serving store, but the Parquet master dataset can contain duplicate rows after a restart. The batch layer's `max(fare)` per trip is deliberately duplicate-insensitive; its row counts are not.
- **No compaction.** A 10-second trigger produces many small Parquet files — the classic small-files problem. Not solved here.
- **The expense source is correlated by construction, not by observation.** It derives plausible costs from the shared roster's baseline duty cycle rather than from the actual GPS track, so the reconciliation variance is realistic in shape but is not a true independent measurement.
- **Security is absent by design** — no TLS, no SASL, no API authentication, default credentials in `docker-compose.yml`.

10.2 What would change at production scale

1. **Storage format.** Replace raw Parquet with Delta Lake or Apache Iceberg: ACID appends, automatic compaction, schema evolution and time travel — the last of which makes “restate that day” a first-class operation rather than a convention.
2. **Schema enforcement at the wire.** Move the telemetry payload from JSON to Avro with a Schema Registry (as built for the Chapter 3 assignment), so a producer cannot silently ship an incompatible field. JSON was chosen here for inspectability during a demo; that is the wrong trade at scale.
3. **Exactly-once.** An idempotent sink keyed on `event_id`, or Kafka transactions spanning the sink write and the offset commit.
4. **Elastic execution.** Spark on Kubernetes or EMR/Databricks, with the speed layer autoscaled on consumer lag rather than fixed at two cores.

5. **Tiered retry topics** instead of in-line retries, so a slow downstream never blocks a partition.
6. **Alertmanager and on-call routing.** Rules currently fire in Prometheus with no destination; production needs routing, grouping, inhibition and silences.
7. **Data-quality as a product.** The validation rules deserve a published contract, a quarantine review workflow and a rule-level SLO, rather than living only as a code module.

11 Conclusion

The platform ingests a continuous telemetry stream and a once-per-simulated-day expense file, processes both through a speed layer and a batch layer, stores an immutable master dataset alongside two clearly-labelled derived views, and answers the use case’s two-part business question through a live dashboard, a REST API and a daily reconciliation report — with structured logging, metrics, correlation identifiers and fifteen alert rules covering every stage.

The architectural claim it rests on is narrow and, I think, defensible: Lambda was selected not because streaming is insufficient, but because this use case has *two different correctness contracts over the same events* — one that must be fast and may be approximate, one that must be exact and restatable against an externally supplied file. Kappa can be made to serve both, but only by simulating batch semantics inside a stream. Lambda’s cost is the duplicated logic, which this implementation reduces by sharing the business rules between layers and then *measures* what remains, rather than claiming to have eliminated it.

Assumptions and simplifications declared. Simulated time is compressed 288× (one simulated day = 300 real seconds, §3.2). Both data sources are simulated Python scripts, and share a deterministic vehicle roster so they describe the same fleet. The downstream “cost partner” is not a real system. Spark runs in local mode; Kafka runs as a single broker with replication factor 1. Monetary values are nominal LKR. *Submission type:* individual.

Repository contents. README.md (architecture summary, setup, the five-minute demo script, troubleshooting), docker-compose.yml (15 services), common/ (shared modules), simulators/, speed_layer/, batch_layer/, serving/, airflow/dags/, sql/, observability/ (Prometheus, 15 alert rules, a 15-panel Grafana dashboard), scripts/ (smoke check, evidence export), tests/ (automated unit tests).